Homework 3

Week 4-5: Fisherface

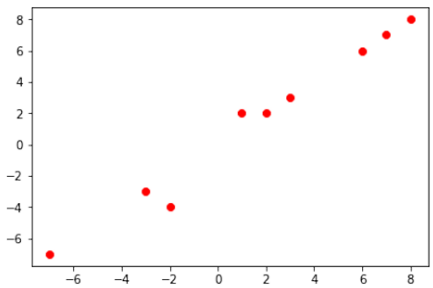
Patthadon Phengpinij
Collaborators. ChatGPT

1 Hello Soft Clustering (GMM)

Recall from HW1 we did K-means clustering. Fitting a GMM on a set of points can be considered as another method to do clustering but now with soft assignments.

Consider the same set of points we used in HW1

x	y
1	2
3	3
2	2
8	8
6	6
7	7
-3	-3
-2	-4
-7	-7



In class, we showed that we could fit a GMM on 1-dimensional data by using Expectation Maximization (EM). The algorithm for doing EM on N-dimensional GMM is very similar. The exact algorithm is as follows:

Initialization: Initialize the mixture weights, $\phi = \{m_j\}$, where j is the mixture number, means of each Gaussian, $\overrightarrow{\mu_j}$ (now a vector of N dimensions), and covariance matrices of each Gaussian, $\mathbf{\Sigma}_j$

Expectation: Find the soft assignments for each data point $w_{n,j}$ where n corresponds to the sample index.

$$w_{n,j} = \frac{p(x_n; \overrightarrow{\mu_j}, \mathbf{\Sigma}_j) m_j}{\sum_j p(x_n; \overrightarrow{\mu_j}, \mathbf{\Sigma}_j) m_j}$$

$w_{n,j}$ means the probability that data point n comes from Gaussian number j.

Maximization: Update the model parameters, ϕ, $\vec{\mu}$, $\mathbf{\Sigma}_j$.

$$m_j = \frac{1}{N}\Sigma_n w_{n,j}$$

$$\vec{\mu_j} = \frac{\Sigma_n w_{n,j}\vec{x_n}}{\Sigma_n w_{n,j}}$$

$$\mathbf{\Sigma}_j = \frac{\Sigma_n w_{n,j}(\vec{x_n} - \vec{\mu_j})(\vec{x_n} - \vec{\mu_j})^T}{\Sigma_n w_{n,j}}$$

The above equation is used for full covariance matrices. For our small toy example, we will use diagonal covariance matrices, which can be acquired by setting the off-diagonal values to zero. In other words, $\mathbf{\Sigma}_{(i,j)} = 0$, for $i \neq j$.

T1. Using 3 mixtures, initialize your Gaussian with means (3,3), (2,2), and (-3,-3), and standard Covariance, **I**, the identity matrix. Use equal mixture weights as the initial weights. Repeat three iterations of EM. Write down $w_{n,j}$, m_j, $\vec{\mu_j}$, $\mathbf{\Sigma}_j$ for each EM iteration. (You may do the calculations by hand or write code to do so)

Solution. The solution goes here.

T2. Plot the log likelihood of the model given the data after each EM step. In other words, plot $\log \prod_n p(\vec{x_n} \mid \phi, \vec{\mu}, \mathbf{\Sigma})$. Does it goes up every iteration just as we learned in class?

Solution. The solution goes here.

T3. Using 2 mixtures, initialize your Gaussian with means (3,3) and (-3,-3), and standard Covariance, **I**, the identity matrix. Use equal mixture weights as the initial weights. Repeat three iterations of EM. Write down $w_{n,j}$, m_j, $\vec{\mu_j}$, $\mathbf{\Sigma}_j$ for each EM iteration.

Solution. The solution goes here.

T4. Plot the log likelihood of the model given the data after each EM step. Compare the log likelihood between using two mixtures and three mixtures. Which one has the better likelihood?

Solution. The solution goes here.

OT1. Using 2 mixtures, initialize your Gaussian with means (0,0) and (10000,10000). Explain what happens. From this case, explain how a proper initialization should be performed. What other tricks can be used to prevent this from happening?

Solution. The solution goes here.

2 The face database

For the rest of the homework we will work on face verification (Given a face, say whether it is person A or not). Face verification is quite related to face recognition (Given a face, say who it is). Face verification is a binary classification task, while face recognition is a multi-class problem.

Download the file `facedata.mat` from github. You can load the data by

```
import scipy.io
data = scipy.io.loadmat(<path to facedata.mat>)
```

data is a dictionary with key value pairs. The data you want to use can be accessed by using 'facedata' as the key.

```
print x["facedata"].shape # face data is a 2-dimensional array
    with size 40x10

# Each data is indexed by i and j
# where i is the person index
# j is the index of the pose
# In other words, there are 40 people in the database.
# There are 10 images per person.
print x["facedata"][0,0]

# Each image is a 56 by 46 image
print x["facedata"][0,0].shape

# You can see the image by using the imshow in matplotlib
import matplotlib.pyplot as plt
import matplotlib.image as mpimg

plt.imshow(x["facedata"][0,0],cmap="gray")
plt.show()
```

Working with images

Each pixel in an image is usually represented by a 8-bit unsigned integer (values from 0 to 255). In order to easily work on images, we usually convert them to floats or doubles using the following command.

```
from skimage import img_as_float

xf = {}
xf[0,0] = img_as_float(x["facedata"][0,0])
print(xf[0,0])
```

`img_as_float` scales 0-255 to 0-1. You can still show the image using the same `imshow` command.

Note that the index of a 2D image starts from the upper left corner of the image. The first dimension goes downwards, while the second dimension goes to the right (think of it as a matrix). To understand what this means, try the following code.

```
plt.imshow(xf[0,0],cmap="gray")
plt.show()
```

```
x_temp = xf[0,0]
x_temp[0:5,0:10] = 1

# In float format, 1 is white
plt.imshow(x_temp[0,0],cmap="gray")
plt.show()
```

The similarity matrix

Consider a set of N data points, a similarity matrix S is a matrix where $S_{i,j}$ is the distance between the ith and the jth data point. A similarity matrix can be very useful for analyzing the data and its distribution. Since a similarity matrix can also be considered as an image, you can also show it as an image to see the pattern in the data.

But how do we define similarity? How can we quantify whether image A is closer to B than image C? One way is to treat each pixel in image as an element in a vector (you may find the function `numpy.reshape()` useful). Then, compare the two vectors using Euclidean distance.

Euclidean distance between vector x and y is defined as:

$$\text{Euclidean Distance} = \sqrt{\Sigma_d (x_d - y_d)^2}$$

where d refers to the index of the dimension.

T5. What is the Euclidean distance between `xf[0,0]` and `xf[0,1]`? What is the Euclidean distance between `xf[0,0]` and `xf[1,0]`? Does the numbers make sense? Do you think these numbers will be useful for face verification?

Solution. The solution goes here.

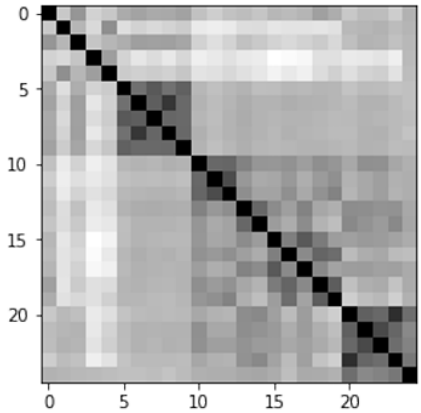
As we continue our exercise, we will refine our feature vectors so that the Euclidean distance between two images can be used in a face verification system.

We define the similarity matrix, A, as a matrix whose elements $A_{i,j}$ is the Euclidean distance between data sample i from list T and data sample j from list D, where list T, D are lists of data samples.

T6. Write a function that takes in a set of feature vectors T and a set of feature vectors D, and then output the similarity matrix A. Show the matrix as an image. Use the feature vectors from the first 3 images from all 40 people for list T (in order $x[0,0], x[0,1], x[0,2], x[1,0], x[1,1], ...x[39,2]$). Use the feature vectors from the remaining 7 images from all 40 people for list D (in order $x[0,3], x[0,4], x[0,5], x[1,6], x[0,7], x[0,8], x[0,9], x[1,3], x[1,4]...x[39,9]$). We will treat T as our training images and D as our testing images.

Solution. The solution goes here.

The picture below shows an example similarity matrix calculated by the first 5 images from the first 5 people (for both T and D).



T7. From the example similarity matrix above, what does the black square between [5:10,5:10] suggest about the pictures from person number 2? What do the patterns from person number 1 say about the images from person 1?

Solution. The solution goes here.

A simple face verification system

In our simple face verification system, given a test image, we want to test if that image comes from person A or not. We will compare the test image against the three training images from person A we have. If the minimum distance (between the three training images) is below a threshold, t, we say that the test image is person A.

T8. Write a function that takes in the similarity matrix created from the previous part, and a threshold t as inputs. The outputs of the function are the true positive rate and the false alarm rate of the face verification task (280 Test images, tested on 40 people, a total of 11200 testing per threshold). What is the true positive rate and the false alarm rate for $t = 10$?

Solution. The solution goes here.

T9. Plot the RoC curve for this simple verification system. What should be the minimum threshold to generate the RoC curve? What should be the maximum threshold? Your RoC should be generated from at least 1000 threshold levels equally spaced between the minimum and the maximum. (You should write a function for this).

Solution. The solution goes here.

T10. What is the EER (Equal Error Rate)? What is the recall rate at 0.1% false alarm rate? (Write this in the same function as the previous question.)

Solution. The solution goes here.

Principle Component Analysis (PCA)

PCA is a method for dimensionality reduction that is very flexible and fits many use cases. It

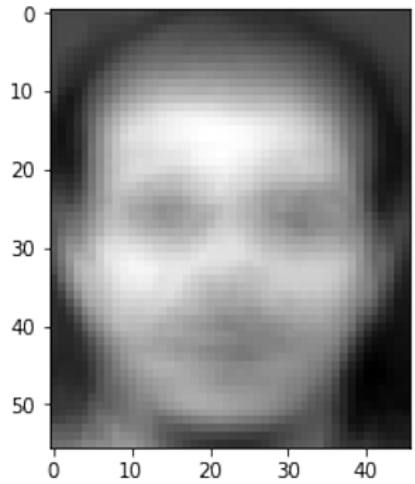
is unsupervised (needs no class label). The core of PCA is using eigendecomposition to decompose the data into the directions of maximum variance.

Let's define a matrix X with each column as an input sample $\vec{x_i}$

A typical PCA starts by normalizing each feature dimension so that they have equal range. For our case, since our input vectors are already between 0 and 1, we can skip this step.

The first step of PCA is to first remove the global mean from our data. Let $\vec{\mu_x}$ be the means of the input data along each input dimension. Let $\hat{X}$ be the matrix with the mean of the input samples removed. Be sure to use the mean computed from just the training examples.

T11. Compute the mean vector from the training images. Show the vector as an image (use `numpy.reshape()`). This is typically called the meanface (or meanvoice for speech signals). You answer should look exactly like the image shown below.



Solution. The solution goes here.

We can then compute eigenvectors on the covariance matrix computed from $\hat{X}$. The PCA vectors would correspond to the eigenvectors, $\vec{v}$. In other words,

$$\Sigma\vec{v} = \lambda\vec{v}$$

However, as learned in class, if we compute the covariance matrix, we would need a lot of space to store it.

T12. What is the size of the covariance matrix? What is the rank of the covariance matrix?

Solution. The solution goes here.

The trick we learned in class is to compute the Gram Matrix $\left(\hat{X}^T\hat{X}\right)$, which is the inner product between the input matrices.

T13. What is the size of the Gram matrix? What is the rank of Gram matrix? If we compute the eigenvalues from the Gram matrix, how many non zero eigenvalues do we expect to get?

Solution. The solution goes here.

T14. Is the Gram matrix also symmetric? Why?

Solution. The solution goes here.

Using the gram matrix, we instead solve for the eigenvector, $\vec{v'}$.

$$\hat{X}^T \hat{X} \vec{v'} = \lambda \vec{v'}$$

where the desired eigenvector (eigenvector of the covariance matrix) can be computed from $\vec{v'}$ (eigenvector of the gram matrix) using the following relationship:

$$\vec{v} = \hat{X} \vec{v'}$$

In order to compute the eigenvectors and eigenvalues, we can use the function `numpy.linalg.eigh` which can be used on symmetric matrices. For symmetric matrices, the eigenvectors and eigenvalues will always be real. In contrast, if the matrix is not symmetric, we have to use the function `numpy.linalg.eig` which will output complex numbers.

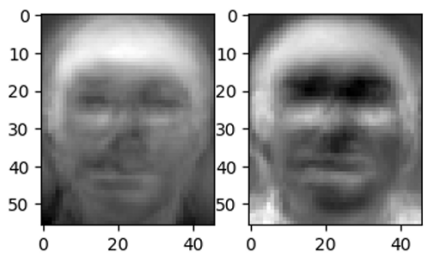
T15. Compute the eigenvectors and eigenvalues of the Gram matrix, $\vec{v'}$ and λ. Sort the eigenvalues and eigenvectors in descending order so that the first eigenvalue is the highest, and the first eigenvector corresponds to the best direction. How many non-zero eigenvalues are there? If you see a very small value, it is just numerical error and should be treated as zero.

Solution. The solution goes here.

T16. Plot the eigenvalues. Observe how fast the eigenvalues decrease. In class, we learned that the eigenvalues is the size of the variance for each eigenvector direction. If I want to keep 95% of the variance in the data, how many eigenvectors should I use?

Solution. The solution goes here.

T17. Compute $\vec{v}$. Don't forget to renormalize so that the norm of each vector is 1 (you can use `numpy.linalg.norm`). Show the first 10 eigenvectors as images. First two example eigenvectors are shown below. We call these images eigenfaces (or eigenvoice for speech signals).



Solution. The solution goes here.

T18. From the image, what do you think the first eigenvector captures? What about the second eigenvector? Look at the original images, do you think biggest variance are capture in these two eigenvectors?

Solution. The solution goes here.

PCA subspace and the face verification system

These eigenfaces we computed serve as good directions to project our data onto in order to decrease the number of dimensions. Since we have shown in class that these eigenvectors are orthogonal (and we normalized them so that they are orthonormal), we can find the projection, $\vec{p}$, of the data onto the eigenface subspace by

$$\vec{p} = \vec{V}^T (\vec{x} - \vec{\mu_x})$$

where V is a matrix whose columns are the eigenvectors, $\vec{v}$. The projection values, $\vec{p}$, will serve as our new input features.

T19. Find the projection values of all images. Keep the first $k = 10$ projection values. Repeat the simple face verification system we did earlier using these projected values. What is the EER and the recall rate at 0.1% FAR?

Solution. The solution goes here.

T20. What is the k that gives the best EER? Try $k = 5, 6, 7, 8, 9, 10, 11, 12, 13, 14$.

Solution. The solution goes here.

(Optional) PCA reconstruction

One of the usage for PCA is compression. Using the project values, we can reconstruct the original image. This can be done by

$$\vec{x'} = \vec{\mu_x} + \sigma_k p_k \vec{v}_k$$

$$\vec{x'} = \vec{\mu_x} + V\vec{p}$$

where $\vec{x'}$ is the reconstructed image.

We can compute the error from such reconstruction by computing the Mean Square Error (MSE)

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(x_i - x'_i)^2$$

where N is the dimension of the original input.

OT2. Reconstruct the first image using this procedure. Use $k = 10$, what is the MSE?

Solution. The solution goes here.

OT3. For k values of 1,2,3,...,10,119, show the reconstructed images. Plot the MSE values.

Solution. The solution goes here.

OT4. Consider if we want to store 1,000,000 images of this type. How much space do we need? If we would like to compress the database by using the first 10 eigenvalues, how much space do we need? (Assume we keep the projection values, the eigenfaces, and the meanface as 32bit floats)

Solution. The solution goes here.

Linear Discriminant Analysis (LDA)

We learned in class that PCA serves well in terms of lowering the dimensionality of the data. However, it does not aim to maximize the classification accuracy. PCA actually aims to retain the most information in the lowest possible subspace (as shown from our reconstruction experiment). PCA is also an unsupervised algorithm. We did not use any class information when we compute for PCA. On the other hand, LDA takes the class labels as inputs and aim to find the projection that maximize the separability between the classes.

LDA is usually used in conjunction with PCA. We first project using PCA to a lower dimensionality then use LDA to project to a subspace that better separates the class.

Assuming everything is already in the PCA subspace, to find the LDA projections, we first need to find the between class scatter, S_B, and the within class scatter, S_W. Between class scatter represents the spread between two classes. In class, for the two class example, it is defined as the distance between the means of class 1 and class 2 as shown below:

$$S_B = (\vec{\mu_1} - \vec{\mu_2})(\vec{\mu_1} - \vec{\mu_2})^T$$

In a multi-class setting, it is defined as the distance of the mean of each class with the global mean, μ:

$$S_B = \sum_{i=1}^{N_c} (\vec{\mu_i} - \vec{\mu})(\vec{\mu_i} - \vec{\mu})^T$$

where N_c is the number of classes.

S_W represents the scatter within each class. For a class i, we can compute the scatter of the class by

$$S_{Wi} = \sum_{j=1}^{N_i} (\vec{x_j} - \vec{\mu_i})(\vec{x_j} - \vec{\mu_i})^T$$

where N_i is the number of data in class i, $\vec{x_j}$ is the jth data sample from class i (in the PCA subspace).

The total within class scatter, S_W, can then be computed by

$$S_W = \sum_{i=1}^{N_c} \sum_{j=1}^{N_i} (\vec{x_j} - \vec{\mu_i})(\vec{x_j} - \vec{\mu_i})^T$$

To find the LDA projection, we want to find a projection, $\vec{w}$, that maximizes S_B, but minimizes S_W. To do so, we maximize the ratio (the Fisher criterion):

$$\frac{\vec{w}^T S_B \vec{w}}{\vec{w}^T S_W \vec{w}}$$

After some calculus, the solution to this maximization is in the form:

$$S_B \vec{w} = \lambda S_W \vec{w}$$

If we assume, S_W is invertible. This becomes

$$S_W^{-1} S_B \vec{w} = \lambda \vec{w}$$

In other words, the LDA projections are the eigenvectors of $S_W^{-1} S_B$.

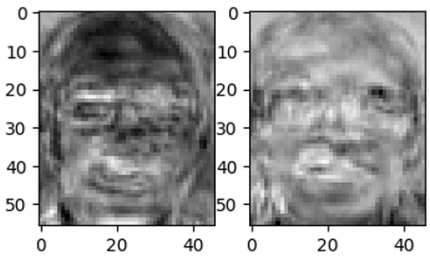
T21. In order to assure that S_W is invertible we need to make sure that S_W is full rank. How many PCA dimensions do we need to keep in order for S_W to be full rank? (Hint: How many dimensions does S_W have? In order to be of full rank, you need to have the same number of linearly independent factors)

Solution. The solution goes here.

T22. Using the answer to the previous question, project the original input to the PCA subspace. Find the LDA projections. To find the inverse, use `numpy.linalg.inv`. Is $S_W^{-1} S_B$ symmetric? Can we still use `numpy.linalg.eigh`? How many non-zero eigenvalues are there?

Solution. The solution goes here.

T23. Plot the first 10 LDA eigenvectors as images (the 10 best projections). Note that in this setup, you need to convert back to the original image space by using the PCA projection. The LDA eigenvectors can be considered as a linear combination of eigenfaces. Compare the LDA projections with the PCA projections. First two example LDA eigenvectors are shown below.



Solution. The solution goes here.

T24. The combined PCA+LDA projection procedure is called fisherface. Calculate the fisherfaces projection of all images. Do the simple face verification experiment using fisherfaces. What is the EER and recall rate at 0.1% FAR?

Solution. The solution goes here.

T25. Plot the RoC of all three experiments (No projection, PCA, and Fisher) on the same axes. Compare and contrast the three results. Submit your writeup and code on MyCourseVille.

Solution. The solution goes here.

OT5. Plot the first two LDA dimensions of the test images from different people (6 people 7 images each). Use a different color for each person. Observe the clustering of between each person. Repeat the same steps for the PCA projections. Does it come out as expected?

Solution. The solution goes here.